the speed at which attackers can guess passwords. Dedicated password cracking machines, which can be built for relatively modest amounts of money, can try many millions or even billions of passwords per second. The expensive time and memory requirements of secure password hashing algorithms like Scrypt reduce this to a few thousand passwords per second, hugely increasing the cost for the attacker and giving users valuable time to change their passwords after a breach is discovered. The latest NIST guidance on secure password storage ("memorized secret verifiers" in the tortured language of NIST) recommends using strong memory-hard hash functions such as Scrypt (https://pages.nist.gov/800-63-3/sp800-63b.html#memsecret).

If you have particularly strict requirements on the performance of authentication to your system, then you can adjust the Scrypt parameters to reduce the time and memory requirements to fit your needs. But you should aim to use the recommended secure defaults until you know that they are causing an adverse impact on performance. You should consider using other authentication methods if secure password processing is too expensive for your application. While there are protocols that allow offloading the cost of password hashing to the client, such as SCRAM[5] or OPAQUE[6], this is hard to do securely so you should consult an expert before implementing such a solution.

> **TIP** ***Establish secure defaults*** for all security-sensitive algorithms and parameters used in your API. Only relax the values if there is no other way to achieve your non-security requirements.